Bloque 2. Despliegue, virtualización y monitorización

En este bloque se estudian los procedimientos necesarios para desplegar aplicaciones web de forma segura, documentar la infraestructura, utilizar contenedores para ejecutar servidores web y analizar los registros de actividad.

> **Objetivo del bloque**
> Desplegar una aplicación web segura, documentar la infraestructura, ejecutar el servidor mediante contenedores y analizar sus registros para comprobar su funcionamiento y detectar posibles problemas.

1. Seguridad avanzada del servidor web (Hardening)

1.1. Concepto de hardening

El **hardening** es el proceso de aplicar medidas de seguridad a un sistema para reducir su superficie de exposición y minimizar las posibilidades de sufrir ataques.

En un servidor web Apache no es suficiente con instalar el servicio y comprobar que muestra una página. También es necesario revisar su configuración, limitar los permisos, reducir la información que proporciona y controlar los accesos.

1.2. Medidas de seguridad en Apache

Ocultar información sobre el servidor

Apache puede proporcionar información sobre su versión y sobre determinados componentes mediante las cabeceras HTTP y las páginas de error. Esta información puede ayudar a identificar posibles vulnerabilidades.

En el archivo de configuración principal o en un archivo independiente:

```
ServerTokens Prod
ServerSignature Off
```

- `ServerTokens Prod`: limita la información de versión que aparece en las cabeceras HTTP.

- `ServerSignature Off`: evita mostrar información del servidor en páginas generadas por Apache.

Desactivar el listado de directorios

Cuando un directorio no contiene un archivo de inicio, Apache puede mostrar un listado de sus archivos si está habilitada la opción correspondiente.

```
<Directory /var/www/html>
    Options -Indexes
    AllowOverride None
    Require all granted
</Directory>
```

Con `Options -Indexes` se evita que los visitantes puedan consultar el listado de archivos de un directorio.

Controlar los permisos

Los archivos de una aplicación web deben tener permisos adecuados. El usuario del servidor web necesita poder leer los archivos públicos, pero no debería tener permisos de escritura innecesarios.

```
sudo chown -R root:www-data /var/www/empresa
sudo find /var/www/empresa -type d -exec chmod 750 {} \;
sudo find /var/www/empresa -type f -exec chmod 640 {} \;
```

En este ejemplo, el propietario es `root` y el grupo es `www-data`, que habitualmente utiliza Apache en Debian.

> **Importante:** Los permisos deben adaptarse a cada aplicación. Algunas necesitan directorios concretos con permisos de escritura para subir archivos, almacenar sesiones o generar caché. No se debe conceder escritura a todo el árbol de la aplicación.

Limitar los métodos HTTP

Los métodos HTTP permiten realizar distintas operaciones sobre los recursos del servidor. Si una aplicación solamente necesita GET y POST, puede limitarse el uso de otros métodos.

```
<Directory /var/www/empresa>
    <LimitExcept GET POST>
        Require all denied
    </LimitExcept>
</Directory>
```

Esta configuración deniega los métodos distintos de GET y POST en el directorio indicado. Debe comprobarse que no interfiera con las necesidades de la aplicación.

1.3. Otras medidas de seguridad

- Mantener actualizado el sistema operativo y Apache.
- Eliminar módulos que no se utilicen.
- Utilizar HTTPS para proteger las comunicaciones.
- Configurar correctamente los permisos de archivos y directorios.
- Limitar los puertos abiertos mediante un cortafuegos.
- Evitar utilizar la cuenta de administrador para ejecutar aplicaciones.
- Realizar copias de seguridad de la configuración y de los datos.
- Revisar periódicamente los registros de actividad.

1.4. Comprobación de la configuración

Antes de aplicar cambios en Apache es recomendable comprobar que los archivos de configuración no contienen errores.

```
sudo apache2ctl configtest
```

Si la configuración es correcta, se mostrará:

```
Syntax OK
```

Para recargar la configuración:

```
sudo systemctl reload apache2
```

Para consultar el estado del servicio:

```
sudo systemctl status apache2
```

Ejemplo: aplicación de medidas de hardening

Se dispone de un servidor Apache que aloja una aplicación en `/var/www/empresa`.

Se realizan las siguientes acciones:

1. Ocultar la versión de Apache.
2. Desactivar la firma del servidor.
3. Deshabilitar el listado de directorios.
4. Revisar los permisos de los archivos.
5. Comprobar la sintaxis de la configuración.
6. Recargar el servicio y verificar el funcionamiento.

2. Documentación técnica del servidor

2.1. Importancia de la documentación

La documentación técnica recoge la información necesaria para comprender, administrar, mantener y recuperar un servidor web. Permite que otros administradores puedan conocer cómo está configurado el sistema sin tener que investigar toda su estructura.

2.2. Información que debe documentarse

Elemento	Información que debe incluirse
Sistema operativo	Distribución, versión y características del servidor.
Servidor web	Versión de Apache, estado y puerto de escucha.
Configuración	Archivos principales y directivas modificadas.
Hosts virtuales	Dominios, directorios raíz y puertos utilizados.
Seguridad	Autenticación, permisos, certificados y restricciones.
Aplicaciones	Ubicación, dependencias, versiones y configuración.
Logs	Ubicación de los registros y procedimientos de consulta.
Mantenimiento	Copias de seguridad, actualizaciones y recuperación.

2.3. Recomendaciones de administración

- Documentar todos los cambios importantes.
- Guardar copias de los archivos de configuración originales.
- Indicar los comandos necesarios para iniciar, detener y reiniciar servicios.
- Registrar las versiones de los componentes instalados.
- No incluir contraseñas ni claves privadas en documentación pública.
- Actualizar la documentación cuando se modifique la infraestructura.

Ejemplo: inventario de un servidor

```
Servidor: servidor-web
Sistema operativo: Debian
Servidor HTTP: Apache 2.4
Puerto HTTP: 80
Puerto HTTPS: 443

Aplicaciones:
- Empresa: /var/www/empresa
- Tienda: /var/www/tienda

Configuración:
- /etc/apache2/apache2.conf
- /etc/apache2/sites-available/

Registros:
- /var/log/apache2/access.log
- /var/log/apache2/error.log

Servicio:
- systemctl start apache2
- systemctl stop apache2
- systemctl reload apache2
```

3. Despliegue de aplicaciones web

3.1. Concepto de despliegue

El despliegue de una aplicación web consiste en preparar el entorno servidor, instalar los componentes necesarios, copiar los archivos de la aplicación, configurar sus dependencias y ponerla a disposición de los usuarios.

Una aplicación desarrollada en un entorno local no siempre puede trasladarse directamente a producción. Es necesario adaptar su configuración al servidor donde se ejecutará.

3.2. Estructura de una aplicación web

Una aplicación web puede contener diferentes tipos de archivos:

- **Archivos públicos:** HTML, CSS, JavaScript e imágenes.
- **Archivos de aplicación:** código PHP, Java u otros lenguajes.

- **Configuración:** parámetros de conexión y variables de entorno.
- **Dependencias:** bibliotecas y componentes externos.
- **Datos:** archivos subidos, documentos y otros recursos.

Una estructura sencilla de una aplicación PHP podría ser:

```
empresa/
├── public/
│   ├── index.php
│   ├── css/
│   ├── js/
│   └── img/
├── src/
├── config/
├── vendor/
├── storage/
└── composer.json
```

El directorio `public` contiene los archivos accesibles desde el navegador. El resto de directorios deben permanecer fuera del acceso público directo siempre que sea posible.

3.3. Instalación de dependencias

Las aplicaciones necesitan determinados componentes para funcionar: intérpretes, módulos, bibliotecas y servicios externos.

Por ejemplo, para ejecutar aplicaciones PHP con Apache en Debian:

```
sudo apt update
sudo apt install apache2 php libapache2-mod-php
```

Si la aplicación utiliza MariaDB y necesita conectarse desde PHP:

```
sudo apt install mariadb-server php-mysql
```

Para comprobar la versión de PHP:

```
php -v
```

Gestión de dependencias con Composer

Composer permite gestionar las bibliotecas externas utilizadas por proyectos PHP. Las dependencias se declaran en el archivo `composer.json`.

```
composer install --no-dev --optimize-autoloader
```

Este comando instala las dependencias necesarias para producción, excluyendo las de desarrollo y optimizando la carga automática.

3.4. Configuración de la aplicación

Una aplicación necesita parámetros que pueden variar dependiendo del entorno en el que se ejecute.

Algunos ejemplos son:

- Dirección del servidor de base de datos.
- Nombre de la base de datos.
- Usuario y contraseña de conexión.
- Modo de funcionamiento: desarrollo o producción.
- Rutas de almacenamiento de archivos.
- Configuración del correo electrónico.

> **Seguridad:** Las credenciales de conexión no deben almacenarse en archivos accesibles desde el navegador ni incluirse directamente en repositorios públicos. Es recomendable utilizar variables de entorno o archivos de configuración protegidos.

3.5. Configuración del VirtualHost

Apache puede utilizar un host virtual para dirigir las peticiones de un dominio hacia el directorio público de una aplicación.

Ejemplo de configuración para `empresa.local`:

```
<VirtualHost *:80>
    ServerName empresa.local

    DocumentRoot /var/www/empresa/public

    <Directory /var/www/empresa/public>
        Options -Indexes
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/empresa_error.log
    CustomLog ${APACHE_LOG_DIR}/empresa_access.log combined
</VirtualHost>
```

Para habilitar el sitio:

```
sudo a2ensite empresa.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

3.6. Comprobación del despliegue

Una vez desplegada la aplicación es necesario comprobar:

- Que Apache está funcionando.
- Que el dominio resuelve correctamente.
- Que la aplicación se muestra en el navegador.
- Que los recursos CSS, JavaScript e imágenes se cargan.
- Que las conexiones a la base de datos funcionan.
- Que los permisos son adecuados.
- Que no aparecen errores en los registros.

Ejemplo: comprobación de errores

```
sudo tail -f /var/log/apache2/error.log
```

El comando permite visualizar los nuevos mensajes que se incorporan al registro de errores mientras se realizan peticiones a la aplicación.

4. Virtualización y contenedores

4.1. Concepto de virtualización

La virtualización permite ejecutar entornos informáticos aislados sobre una misma infraestructura física. Cada entorno puede disponer de su propio sistema operativo, aplicaciones y configuración.

Existen diferentes tecnologías:

- **Máquinas virtuales:** ejecutan un sistema operativo completo sobre un hipervisor.
- **Contenedores:** ejecutan aplicaciones aisladas compartiendo el núcleo del sistema operativo anfitrión.

4.2. Máquinas virtuales y contenedores

Característica	Máquina virtual	Contenedor
Sistema operativo	Incluye un sistema operativo completo.	Comparte el núcleo del anfitrión.
Consumo	Generalmente mayor.	Generalmente menor.
Arranque	Más lento.	Más rápido.
Aislamiento	A nivel de máquina virtual.	A nivel de procesos y recursos.
Uso habitual	Entornos completos y sistemas operativos diferentes.	Despliegue de aplicaciones y servicios.

4.3. Introducción a Docker

Docker es una plataforma que permite crear, distribuir y ejecutar aplicaciones mediante contenedores.

Un contenedor incorpora la aplicación y los componentes necesarios para ejecutarla, lo que facilita reproducir el entorno en diferentes equipos y servidores.

Conceptos fundamentales

- **Imagen:** plantilla que contiene el sistema de archivos y los componentes necesarios para ejecutar un servicio.
- **Contenedor:** instancia ejecutable de una imagen.
- **Dockerfile:** archivo que define cómo construir una imagen.
- **Volumen:** mecanismo para conservar datos fuera del ciclo de vida del contenedor.
- **Red:** permite la comunicación entre contenedores y otros sistemas.
- **Registro:** repositorio donde se almacenan y distribuyen imágenes.

4.4. Comandos básicos de Docker

Descargar una imagen de Apache:

```
docker pull httpd
```

Consultar las imágenes disponibles:

```
docker images
```

Crear y ejecutar un contenedor:

```
docker run -d \
    --name servidor-web \
    -p 8080:80 \
    httpd
```

En este ejemplo:

- `-d` : ejecuta el contenedor en segundo plano.
- `--name` : asigna un nombre al contenedor.
- `-p 8080:80` : relaciona el puerto 8080 del anfitrión con el puerto 80 del contenedor.

- `httpd` : imagen utilizada.

Para acceder al servidor se utiliza: `http://localhost:8080`.

Consultar y administrar contenedores

```
# Ver contenedores en ejecución
docker ps

# Ver todos los contenedores
docker ps -a

# Detener un contenedor
docker stop servidor-web

# Iniciar un contenedor
docker start servidor-web

# Reiniciar un contenedor
docker restart servidor-web

# Eliminar un contenedor detenido
docker rm servidor-web

# Consultar los registros
docker logs servidor-web
```

4.5. Despliegue de una aplicación mediante volúmenes

Los volúmenes y montajes permiten compartir archivos entre el sistema anfitrión y un contenedor. De esta forma se pueden modificar los archivos de una aplicación sin tener que construir una nueva imagen cada vez.

Por ejemplo, si se dispone de una aplicación HTML en el directorio `./web`:

```
docker run -d \
    --name web-empresa \
    -p 8080:80 \
    -v ./web:/usr/local/apache2/htdocs:ro \
    httpd
```

El contenido del directorio `web` del anfitrión se muestra desde el servidor Apache del contenedor. La opción `:ro` monta los archivos en modo lectura.

4.6. Construcción de imágenes con Dockerfile

Un Dockerfile permite definir los pasos necesarios para construir una imagen personalizada.

Ejemplo: imagen Apache con una aplicación web

Estructura del proyecto:

```
proyecto/
├── Dockerfile
└── web/
    ├── index.html
    └── estilos.css
```

Contenido del Dockerfile:

```
FROM httpd:2.4

COPY ./web/ /usr/local/apache2/htdocs/

EXPOSE 80
```

Construir la imagen:

```
docker build -t empresa-web .
```

Ejecutar el contenedor:

```
docker run -d \
    --name empresa \
    -p 8080:80 \
    empresa-web
```

Al acceder a `http://localhost:8080` se mostrará la página web incluida en la imagen.

4.7. Redes de Docker

Las redes permiten que varios contenedores se comuniquen entre sí. Por ejemplo, una aplicación web puede necesitar comunicarse con un contenedor que ejecuta MariaDB.

Crear una red:

```
docker network create red-empresa
```

Consultar las redes disponibles:

```
docker network ls
```

Los contenedores conectados a una misma red pueden comunicarse utilizando sus nombres de servicio o contenedor, según la configuración de la red.

4.8. Docker Compose

Docker Compose permite definir y gestionar varios servicios relacionados mediante un archivo YAML, normalmente denominado `compose.yaml`.

En lugar de ejecutar varios comandos independientes, se describe la infraestructura completa en un único archivo.

Ejemplo: Apache y MariaDB con Docker Compose

Estructura del proyecto:

```
empresa/
├── compose.yaml
└── web/
    └── index.html
```

Archivo `compose.yaml` :

```
services:

  web:
    image: httpd:2.4
    container_name: empresa-web
    ports:
      - "8080:80"
    volumes:
      - ./web:/usr/local/apache2/htdocs:ro
    depends_on:
      - db

  db:
    image: mariadb:11
    container_name: empresa-db
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MARIADB_DATABASE: empresa
      MARIADB_USER: empresa
      MARIADB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - datos_db:/var/lib/mysql

volumes:
  datos_db:
```

Las contraseñas se obtienen de variables de entorno, por ejemplo desde un archivo `.env` que no debe publicarse en el repositorio.

Para iniciar los servicios:

```
docker compose up -d
```

Consultar su estado:

```
docker compose ps
```

Consultar los registros:

```
docker compose logs
```

Detener los servicios:

```
docker compose down
```

Importante: En producción no deben utilizarse contraseñas sencillas ni publicarse archivos `.env`. También es necesario gestionar las actualizaciones de imágenes, los permisos, las copias de seguridad y la persistencia de los datos. El ejemplo utiliza una imagen de MariaDB para ilustrar la organización de servicios; Apache y MariaDB necesitan configuración adicional para que una aplicación PHP pueda comunicarse con la base de datos.

4.9. Introducción al despliegue en la nube

La computación en la nube permite utilizar recursos informáticos proporcionados por un proveedor externo, sin necesidad de disponer de toda la infraestructura física.

Entre los servicios habituales se encuentran:

- **IaaS:** infraestructura como servicio. Proporciona máquinas virtuales, redes y almacenamiento.
- **PaaS:** plataforma como servicio. Ofrece un entorno preparado para desplegar aplicaciones.
- **SaaS:** software como servicio. Proporciona aplicaciones completas accesibles a través de Internet.
- **Contenedores gestionados:** permiten ejecutar aplicaciones contenerizadas sin administrar directamente todos los servidores.

Un despliegue en la nube puede incluir máquinas virtuales, contenedores, bases de datos gestionadas, almacenamiento externo, balanceadores de carga y servicios de monitorización.

5. Gestión y monitorización de logs

5.1. Concepto de log

Un log o registro es un archivo o conjunto de datos donde se almacenan acontecimientos producidos durante la ejecución de un sistema o aplicación.

Los registros permiten conocer qué ha sucedido en el servidor, cuándo se ha producido un error, qué usuarios han realizado peticiones y cómo ha respondido la aplicación.

5.2. Logs de Apache

Apache utiliza principalmente dos tipos de registros:

Registro	Contenido	Ubicación habitual en Debian
Access log	Peticiones recibidas, direcciones IP, recursos solicitados y códigos HTTP.	`/var/log/apache2/access.log`
Error log	Errores de ejecución, problemas de configuración y advertencias.	`/var/log/apache2/error.log`

5.3. Consultar los registros

Linux proporciona diferentes herramientas para consultar y analizar archivos de registro.

Visualizar el contenido

```
cat /var/log/apache2/access.log
```

Consultar las últimas líneas

```
tail -n 20 /var/log/apache2/access.log
```

Monitorizar en tiempo real

```
tail -f /var/log/apache2/access.log
```

Esta última opción permite observar las nuevas peticiones que recibe Apache mientras se utiliza la aplicación.

Buscar errores HTTP

```
grep "404" /var/log/apache2/access.log
```

Este comando busca las líneas que contienen el código 404, que indica que el recurso solicitado no se ha encontrado.

5.4. Códigos de estado HTTP

Los códigos HTTP permiten identificar el resultado de una petición. Son especialmente útiles para analizar los registros de acceso.

Familia	Significado	Ejemplos
1xx	Respuestas informativas.	100 Continue
2xx	Petición procesada correctamente.	200 OK, 201 Created
3xx	Redirecciones.	301 Moved Permanently, 302 Found
4xx	Errores relacionados con la petición del cliente.	403 Forbidden, 404 Not Found
5xx	Errores internos del servidor.	500 Internal Server Error, 503 Service Unavailable

5.5. Configuración de registros por sitio virtual

Cuando Apache aloja diferentes sitios virtuales, es recomendable mantener registros independientes para cada uno.

```
<VirtualHost *:80>
    ServerName empresa.local
    DocumentRoot /var/www/empresa/public

    ErrorLog ${APACHE_LOG_DIR}/empresa_error.log
    CustomLog ${APACHE_LOG_DIR}/empresa_access.log combined
</VirtualHost>
```

De esta forma es posible analizar la actividad de cada aplicación de manera independiente.

5.6. Rotación de logs

Los registros pueden crecer continuamente y ocupar grandes cantidades de espacio en disco. La rotación de logs permite administrar su tamaño y conservar un histórico de actividad.

En Debian se utiliza habitualmente `logrotate`. Apache suele disponer de una configuración en:

```
/etc/logrotate.d/apache2
```

Un ejemplo simplificado de configuración sería:

```
/var/log/apache2/*.log {
    weekly
    rotate 8
    compress
    delaycompress
    missingok
    notifempty
    sharedscripts
    postrotate
        if systemctl is-active --quiet apache2; then
            systemctl reload apache2
        fi
    endscript
}
```

En este ejemplo:

- `weekly`: realiza la rotación semanalmente.
- `rotate 8`: conserva ocho archivos rotados.
- `compress`: comprime los registros antiguos.
- `delaycompress`: retrasa la compresión del último archivo rotado.
- `missingok`: no genera errores si el archivo no existe.
- `notifempty`: no rota archivos vacíos.

5.7. Monitorización del servidor

La monitorización consiste en observar continuamente el estado y el comportamiento de un sistema para detectar problemas, medir su rendimiento y anticiparse a posibles incidencias.

Algunos parámetros importantes son:

- Uso de CPU.
- Memoria RAM disponible.
- Espacio libre en disco.
- Número de conexiones activas.
- Peticiones HTTP recibidas.
- Tiempo de respuesta.
- Errores HTTP 4xx y 5xx.
- Disponibilidad del servicio.

Herramientas básicas de Linux

```
# Uso de memoria
free -h

# Espacio disponible
df -h

# Procesos en ejecución
top

# Estado del servicio Apache
systemctl status apache2

# Conexiones de red
ss -tulpen
```

5.8. Herramientas de análisis de logs

Cuando un servidor genera miles o millones de registros, la consulta manual deja de ser suficiente. Existen herramientas que permiten recopilar, procesar, visualizar y analizar grandes volúmenes de datos.

Herramienta	Funcionalidad
GoAccess	Analiza registros de servidores web y genera informes estadísticos y visualizaciones.
ELK Stack	Elasticsearch, Logstash y Kibana permiten almacenar, procesar y visualizar registros.
Prometheus	Recopila y almacena métricas para monitorizar sistemas y servicios.
Grafana	Permite crear paneles de visualización de métricas y datos.

5.9. Análisis de logs con GoAccess

GoAccess es una herramienta de análisis de registros web que permite obtener información estadística sobre el uso de un servidor Apache.

Instalación en Debian:

```
sudo apt update
sudo apt install goaccess
```

Generar un informe en HTML:

```
sudo goaccess \
    /var/log/apache2/access.log \
    --log-format=COMBINED \
    -o /var/www/html/informe.html
```

El archivo generado contiene información sobre:

- Número de peticiones realizadas.
- Visitantes y direcciones IP.
- Recursos más solicitados.
- Códigos de estado HTTP.
- Navegadores y sistemas operativos.
- Distribución temporal de las peticiones.

> **Precaución:** Los informes de análisis pueden contener información sensible sobre la actividad de los usuarios. No deben publicarse sin aplicar las medidas de acceso y protección adecuadas.

5.10. Big Data y análisis de registros

En infraestructuras grandes, los servidores pueden generar una cantidad de registros que requiere herramientas especializadas para su almacenamiento y análisis.

El análisis de grandes volúmenes de logs permite:

- Identificar patrones de acceso.
- Detectar comportamientos anómalos.
- Localizar errores frecuentes.
- Analizar la evolución del tráfico.
- Identificar recursos con mayor demanda.
- Mejorar la capacidad y el rendimiento de la infraestructura.

Por ejemplo, un incremento repentino de respuestas HTTP 500 puede indicar un problema en la aplicación. Un elevado número de respuestas 404 puede señalar enlaces incorrectos o intentos de localizar recursos inexistentes.

> **Ejemplo: análisis de incidencias**
>
> Un administrador observa que el número de errores 500 aumenta durante las horas de mayor actividad.
>
> 1. Consulta los registros de acceso para localizar las peticiones afectadas.
> 2. Revisa el registro de errores de Apache.
> 3. Comprueba el estado de los procesos y el uso de memoria.
> 4. Identifica si el problema está relacionado con la aplicación, la base de datos o los recursos del servidor.
> 5. Aplica las medidas correctoras y comprueba si disminuyen los errores.

---

6. Ejercicios prácticos de repaso

Los siguientes ejercicios permiten practicar los principales procedimientos estudiados en este bloque.

Ejercicio 1. Aplicación de medidas de seguridad

Sobre un servidor Apache instalado en Debian, realiza una revisión de seguridad que incluya la ocultación de información del servidor, la desactivación del listado de directorios y la revisión de los permisos de una aplicación web.

Comprueba que Apache sigue funcionando correctamente después de aplicar los cambios.

Ejercicio 2. Documentación técnica

Elabora un inventario técnico de un servidor web que incluya información del sistema operativo, versión de Apache, hosts virtuales, directorios de aplicaciones, configuración de seguridad, certificados, registros y comandos de administración.

Ejercicio 3. Despliegue de una aplicación web

Despliega una aplicación web en Apache utilizando un VirtualHost propio. Configura su directorio público, instala las dependencias necesarias y comprueba su funcionamiento desde un navegador.

Ejercicio 4. Despliegue mediante Docker

Crea un Dockerfile para generar una imagen personalizada de Apache que incluya una aplicación web. Construye la imagen, ejecuta un contenedor y comprueba el acceso desde el navegador.

Ejercicio 5. Infraestructura con Docker Compose

Define mediante Docker Compose una infraestructura formada por un servidor web y una base de datos MariaDB. Configura los servicios, la comunicación entre ellos y la persistencia de los datos.

Ejercicio 6. Análisis de registros

Genera diferentes peticiones a un servidor Apache, incluyendo accesos correctos y solicitudes a recursos inexistentes. Consulta los registros, identifica los códigos HTTP y genera un informe estadístico utilizando GoAccess.

7. Resumen de conceptos fundamentales

Concepto	Descripción
Hardening	Aplicación de medidas para reducir las vulnerabilidades del servidor.
Documentación técnica	Registro de la configuración, administración y mantenimiento del sistema.
Despliegue	Preparación e instalación de una aplicación en un entorno servidor.
Virtualización	Creación de entornos informáticos aislados sobre una infraestructura física.
Imagen Docker	Plantilla utilizada para crear contenedores.
Contenedor	Instancia ejecutable y aislada de una imagen.
Dockerfile	Archivo que define cómo construir una imagen.
Docker Compose	Herramienta para definir y gestionar aplicaciones con varios servicios.
Volumen	Mecanismo para conservar datos fuera del ciclo de vida del contenedor.
Log	Registro de acontecimientos producidos por un sistema o aplicación.
Rotación de logs	Proceso de administración y conservación de registros.
Monitorización	Supervisión del estado, rendimiento y disponibilidad del servidor.
GoAccess	Herramienta para analizar registros de servidores web.
Big Data	Tratamiento y análisis de grandes volúmenes de datos.